Blocks - backend handoff

Scope: the freelancer **Blocks** screen only (`/freelancer/blocks`). **Audience:** backend developer. **Summary:** the front-end for the new "Blocks builder" is built and working against **localStorage** (so it demos today). It now needs real endpoints to persist. This doc says what already exists, what's new, and the exact data + pricing model to build.

---

1. What you've already built (the existing `blocks` API)

This still stands - don't change it. It models a **dated run of a listing**:

- `blocks` collection: `{ tenantId, listingId, name, startDate, endDate, capacity, bookedCount, open, sessions[] }`
- Endpoints: `GET/POST /api/blocks`, `PUT/DELETE /api/blocks/:id`, `GET /api/blocks/:id/attendees`
- Capacity + waitlist enforced in the booking flow; delete returns **409** if a block has bookings.

Nothing about that changes. The new work below is a *separate* feature that happens to share the word "block".

---

2. What's new (and why)

The Blocks **builder** is a different concept from a dated run. Operators build **reusable scheduling patterns** once and reuse them across listings:

- **Periods** - named session time windows (Standard day, Early drop-off, Late pick-up...).
- **Passes** - how long a parent books (1-day, 5-day week...).
- **Block bundles** - a named set of periods + passes, kept in a **Block Library**

(search, reorder, archive, duplicate), with a **pricing calculator**, that can be **sent to one or more listings**.

None of this exists in the backend yet. Three new tenant-scoped collections + endpoints are needed. **Reuse the exact tenancy/role rules from** `blocks`/`listings` (operator-only writes, scope derived from the signed-in account, platform read via `?tenantId=`, cross-tenant -> 404).

> *[!]* ***Name clash:*** *the existing* `blocks` *collection is the dated-run model. The builder's "block" is a bundle - use a* ***different*** *collection name. This doc uses* `blockBundles`*.*

---

3. New collections

`periods`

field	type	notes
`tenantId`	string	owner
`title`	string (2-120)	"Early drop-off"
`start`	string `"HH:MM"`	24h
`finish`	string `"HH:MM"`	24h, after `start`

`passes` ***(distinct from*** `listing.passes`***, which are priced rows on a listing)***

field	type	notes
`tenantId`	string	
`name`	string (2-120)	"5-day week pass"
`days`	integer >= 1	length booked

`blockBundles`

field	type	notes
`tenantId`	string	
`name`	string (2-120)	"Summer Multi-Activity Camp"
`periodIds`	string[]	refs into `periods` (same tenant)
`passIds`	string[]	refs into `passes` (same tenant)
`listingIds`	string[]	listings this bundle is sent to (0+)
`order`	integer	position in the library (drag-to-reorder)
`archived`	boolean	archived bundles hide from the main list
`priced`	boolean	true once a master price is set
`masterPrice`	number \	null
`calcOn`	boolean (default true)	auto-calculate on/off
`passFlat`	map `passId -> number`	per-pass price overrides
`passMode`	map `passId -> "flat"`	which passes are overridden
`periodPrice`	map `"{passId}_{periodId}" -> number`	per-pass, per-period overrides

---

4. The pricing model (build this exactly - it's the tricky bit)

The operator sets **one** number: the full price of the **longest** pass. Everything else is derived, and any value can be overridden.

```
passesSorted = passIds resolved, sorted by days DESC
master       = passesSorted[0]                       // the longest pass
perDay       = calcOn ? masterPrice / master.days : 0

hours(period)   = (finishMins - startMins) / 60, min 1
baseHours(bund) = max hours across the bundle's periods (1 if none)

priceForPass(q, idx):
  if idx == 0:            return masterPrice          // the master IS the price
  if !calcOn:             return passFlat[q.id] ?? 0
  if passMode[q.id]=='flat': return passFlat[q.id] ?? 0
  else:                  return q.days * perDay        // auto

priceForTiming(pass p, period r):           // shown per pass, per period
  override = periodPrice["{p.id}_{r.id}"]
  if override != null:   return override
  return calcOn ? passPrice(p) * hours(r) / baseHours : 0
```

Worked example - 5-day week pass = **£160** (longest): perDay = £32 -> 4-day = **£128**, 1-day = **£32**. For the 5-day pass: Standard day (6.5h, the longest period) = **£160**, Late pick-up (2h) = 160 x 2 / 6.5 = **£49.23**.

The front-end computes this today; the server only needs to store the fields above. BUT the storefront/checkout (parent booking) will need the resolved prices - see open question 1.

---

5. Endpoints (mirror `blocks.ts` conventions)

```
GET    /api/periods                      -&gt; Period[]
POST   /api/periods                      -&gt; 201 Period
PUT    /api/periods/:id                  -&gt; 200 Period
DELETE /api/periods/:id                  -&gt; 200 { ok }   // cascade: strip id from every bundle.periodIds

GET    /api/passes                       -&gt; Pass[]
POST   /api/passes                       -&gt; 201 Pass
PUT    /api/passes/:id                   -&gt; 200 Pass
DELETE /api/passes/:id                   -&gt; 200 { ok }   // cascade: strip id from every bundle.passIds

GET    /api/block-bundles                -&gt; BlockBundle[]  (sorted by `order`)
POST   /api/block-bundles                -&gt; 201 BlockBundle
```

```
PUT    /api/block-bundles/:id              -&gt; 200 BlockBundle
DELETE /api/block-bundles/:id              -&gt; 200 { ok }
POST   /api/block-bundles/:id/duplicate    -&gt; 201 BlockBundle          // deep copy, name "... (copy)", listingId
POST   /api/block-bundles/:id/archive      -&gt; 200 BlockBundle          // { archived: true|false } to un/archive
POST   /api/block-bundles/reorder          -&gt; 200 { ok }   body { orderedIds: string[] }
PUT    /api/block-bundles/:id/listings     -&gt; 200 BlockBundle          // body { listingIds: string[] } (send-to-
```

Errors follow the existing shape: `401` unauth, `403` non-operator, `404` cross-tenant/missing, `400` validation.

Schemas (OpenAPI-style, to fold into `server/openapi.yaml`)

```
Period:
  type: object
  properties:
    id: { type: string }
    title: { type: string }
    start: { type: string, example: "08:00" }
    finish: { type: string, example: "09:00" }
Pass:
  type: object
  properties:
    id: { type: string }
    name: { type: string, example: "5-day week pass" }
    days: { type: integer, example: 5 }
BlockBundle:
  type: object
  properties:
    id: { type: string }
    name: { type: string }
    periodIds: { type: array, items: { type: string } }
    passIds: { type: array, items: { type: string } }
    listingIds: { type: array, items: { type: string } }
    order: { type: integer }
    archived: { type: boolean }
    priced: { type: boolean }
    masterPrice: { type: number, nullable: true }
    calcOn: { type: boolean }
    passFlat: { type: object, additionalProperties: { type: number } }
    passMode: { type: object, additionalProperties: { type: string, enum: [flat] } }
    periodPrice: { type: object, additionalProperties: { type: number } }
```

---

6. Open questions (need product/you to decide)

1. **Where do the resolved prices go for booking?** The calculator lives in the builder; the storefront needs the actual pass/timing prices at checkout. Either (a) the server replicates the §4 formula and exposes resolved prices, or (b) `send-to-listings` snapshots the computed prices onto the listing. Pick one.
2. **What does "send to a listing" actually do** beyond setting `listingIds`? Just an association, or does it also **materialise dated runs** in the existing `blocks` collection (turn periods + a date range into bookable sessions/capacity)? This is where the builder meets your existing dated-run model.
3. **Passes vs** `listing.passes` - should sending a bundle create/refresh the priced `passes` rows on the listing?
4. **Reorder** - is a per-bundle `order` int fine (this doc's approach), or would you rather the client just PUT the whole ordered list?

---

7. Front-end contract (so payloads line up)

The TypeScript types in `features/blocks/BlocksApp.tsx` (`Period`, `Pass`, `LibraryBlock` = `BlockBundle`) already match the schemas above. To switch off localStorage: replace the `load()`/`save()` helpers with the endpoints in §5 (via `lib/api.ts`), and add `periods` / `passes` / `blockBundles` to the realtime stream (`server/src/routes/events.ts`) so the view live-refreshes. No other UI changes needed.